

Optimizing Defensive Assignments in Co-Ed Slowpitch Softball

David Russo

July 2026

Abstract

This paper describes an optimization model for assigning available players to defensive positions during a seven-inning co-ed slowpitch softball game. A valid schedule must obey league rules about the number of fielders and the number and placement of women, while also respecting each player's stated position preferences. Among valid schedules, the model first seeks fair playing time. It then tries to keep players at a small number of positions and to give them continuous position assignments lasting two or more innings, preferably three or more. The problem is formulated as a binary integer constraint program and solved with Google's OR-Tools CP-SAT solver. A two-stage objective separates playing-time fairness from positional consistency. The resulting Python program is exposed through a mobile-friendly Streamlit website so a manager can update last-minute availability and obtain a new schedule in about three seconds.

1 Problem Statement

A slowpitch softball defense normally uses ten positions: pitcher (P), catcher (C), first base (1B), second base (2B), third base (3B), shortstop (SS), left field (LF), left center field (LC), right center field (RC), and right field (RF). A roster may contain more than ten available players, so some players must sit out in each inning. The manager must decide who plays, who sits, and which position every fielder plays in each of seven innings.

The co-ed rules make this more than a simple rotation. A ten-player defense requires at least four women. It must also include at least one woman in the infield and at least one woman in the outfield. If only three women are available, the team may use at most nine fielders, leaving RF empty. With only eight total players, both RF and C are empty. Both eight- and nine-player lineups require at least three women, including at least one in the infield and one in the outfield. Pitcher and catcher count as infield positions.

Players also have unranked positional preferences. In this application a preference is a hard eligibility rule: a player cannot be assigned to a position that the player did not select. The one exception occurs when the team fields eight or nine players. Because RF is then removed, a player who selected RF is also considered eligible for RC. This exception gives a right fielder a nearby position without weakening preferences in a normal ten-player lineup.

A schedule should satisfy three practical goals in priority order:

1. Every inning must be legal and every occupied position must be filled by an eligible player.
2. Playing time should be as equal as the rules and preferences permit.
3. Position assignments should be stable. A player should use few distinct positions and should remain at a newly assigned position for at least two consecutive innings, preferably three or more.

These goals can conflict. For example, equal playing time might require a specialist to enter for one inning, even though a longer assignment would be more consistent. Therefore legality and preferences are modeled as hard constraints, while fairness and consistency are modeled as ordered optimization goals. The model optimizes one game at a time because availability can change shortly before a game.

2 Sets, Data, and Lineup Size

Let $\mathcal{P}$ be the set of available players and let $N = |\mathcal{P}|$. Let $\mathcal{W} \subseteq \mathcal{P}$ be the set of available women. The seven innings are $\mathcal{I} = \{1, \dots, 7\}$. The full position set is

$$\mathcal{S} = \{P, C, 1B, 2B, 3B, SS, LF, LC, RC, RF\}.$$

The infield and outfield subsets are

$$\mathcal{S}_{IF} = \{P, C, 1B, 2B, 3B, SS\}, \quad \mathcal{S}_{OF} = \{LF, LC, RC, RF\}.$$

The lineup size K and active position set $\mathcal{S}_K$ are computed before the optimization model is built. This is useful because the league rule determines K directly from availability; the solver does not need to choose it.

K	Active positions $\mathcal{S}_K$	Minimum women g_K
10	all ten positions	4
9	all positions except RF	3
8	all positions except RF and C	3

The program selects $K = 10$ if $N \geq 10$ and at least four women are available. Otherwise it selects $K = 9$ if $N \geq 9$ and at least three women are available. Otherwise it selects $K = 8$ if $N = 8$ and at least three women are available. Fewer than eight players or fewer than three women produce an infeasibility message before the solver is called.

For each player p , let $E_p \subseteq \mathcal{S}_K$ be that player's eligible active positions. It is the intersection of the player's preferences and the active position set. When $K \in \{8, 9\}$ and RF was selected, RC is added to E_p . A preflight check reports an error if no eligible player exists for a required position or if no available woman is eligible for an active infield or outfield position.

3 Decision Variables

The main decision variable is

$$x_{ips} = \begin{cases} 1, & \text{if player } p \text{ plays position } s \text{ in inning } i, \\ 0, & \text{otherwise.} \end{cases}$$

It is defined for $i \in \mathcal{I}$, $p \in \mathcal{P}$, and $s \in E_p$. Variables for non-preferred positions are omitted completely, which reduces model size and makes preference violations impossible.

Several auxiliary variables measure the quality of a schedule:

- $t_p \in \{0, \dots, 7\}$ is the total innings played by player p .
- y_{ps} is 1 if player p uses position s at least once.
- $u_p = \sum_s y_{ps}$ is the number of distinct positions used by player p .
- $e_p = \max(u_p - 2, 0)$ counts positions beyond the preferred soft limit of two.
- a_{ips} is 1 when player p starts a new consecutive stint at position s in inning i .
- $q_{ips}^{(1)}$ is 1 when inning i is an entire one-inning stint for player p at position s .
- $q_{ips}^{(2)}$ is 1 when inning i begins an exact two-inning stint for player p at position s .

The model also uses $t^{\max}$ and $t^{\min}$ for the largest and smallest individual playing-time totals. Their difference $R = t^{\max} - t^{\min}$ is the playing-time range.

4 Hard Constraints

4.1 Fill every active position

Every active position must have exactly one player in every inning:

$$\sum_{\substack{p \in \mathcal{P} \\ s \in E_p}} x_{ips} = 1 \quad \forall i \in \mathcal{I}, s \in \mathcal{S}_K.$$

This equation also implies that exactly K players field in each inning.

4.2 Use each player at most once per inning

A person cannot occupy two positions in the same inning:

$$\sum_{s \in E_p} x_{ips} \leq 1 \quad \forall i \in \mathcal{I}, p \in \mathcal{P}.$$

Because x_{ips} only exists for $s \in E_p$, this constraint works together with the variable definition to enforce positional preferences.

4.3 Meet the co-ed requirements

Each inning must contain the required minimum number of women:

$$\sum_{p \in \mathcal{W}} \sum_{s \in E_p} x_{ips} \geq g_K \quad \forall i \in \mathcal{I}.$$

At least one woman must be in each defensive region:

$$\sum_{p \in \mathcal{W}} \sum_{s \in E_p \cap \mathcal{S}_{IF}} x_{ips} \geq 1 \quad \forall i \in \mathcal{I},$$

$$\sum_{p \in \mathcal{W}} \sum_{s \in E_p \cap \mathcal{S}_{OF}} x_{ips} \geq 1 \quad \forall i \in \mathcal{I}.$$

These are lower bounds rather than equalities, so a lineup may use more women than the minimum.

4.4 Define playing time and position use

The innings played by each person are

$$t_p = \sum_{i \in \mathcal{I}} \sum_{s \in E_p} x_{ips}.$$

The largest and smallest values satisfy $t^{\max} = \max_p t_p$ and $t^{\min} = \min_p t_p$. CP-SAT supports these maximum and minimum relationships directly.

Position-use variables are linked to assignments by

$$x_{ips} \leq y_{ps} \quad \forall i, p, s, \quad y_{ps} \leq \sum_{i \in \mathcal{I}} x_{ips} \quad \forall p, s.$$

Thus y_{ps} equals 1 exactly when the position appears in that player's schedule. The excess variable obeys $e_p \geq u_p - 2$ and $e_p \geq 0$.

4.5 Identify consecutive position stints

A stint is an uninterrupted run at the same position. Sitting out ends a stint, even if the player later returns to the same position. For a start variable after the first inning, the following linear inequalities represent the logical statement “assigned now and not assigned here previously”:

$$a_{ips} \geq x_{ips} - x_{i-1,p,s}, \quad a_{ips} \leq x_{ips}, \quad a_{ips} \leq 1 - x_{i-1,p,s}.$$

For the first inning, $a_{1ps} = x_{1ps}$.

For one-inning stints, define the boundary values $x_{0ps} = x_{8ps} = 0$. Then $q_{ips}^{(1)}$ represents the pattern 0, 1, 0:

$$\begin{aligned} q_{ips}^{(1)} &\leq x_{ips}, \\ q_{ips}^{(1)} &\leq 1 - x_{i-1,p,s}, \\ q_{ips}^{(1)} &\leq 1 - x_{i+1,p,s}, \\ q_{ips}^{(1)} &\geq x_{ips} - x_{i-1,p,s} - x_{i+1,p,s}. \end{aligned}$$

Similar “if and only if” inequalities identify an exact two-inning pattern 0, 1, 1, 0. These variables do not prohibit short stints. Instead, they allow the objective function to penalize them while preserving feasibility when fairness or hard preferences make a short assignment unavoidable.

5 Objective Function

There is no single natural unit that compares fairness with consistency. A one-inning stint, for example, cannot be directly compared with one inning of playing-time imbalance. The implementation handles this by solving two objectives in sequence.

5.1 Stage 1: playing-time fairness

There are $H = 7K$ total fielding slots in a game. The perfectly equal share is H/N innings per player, which is usually fractional. CP-SAT uses integer expressions, so the model defines the scaled absolute deviation

$$D_p = |Nt_p - H| = N \left| t_p - \frac{H}{N} \right|.$$

Multiplying by N avoids fractions without changing which schedule is best. Let D^{UB} be a safe upper bound on $\sum_p D_p$. The first objective is

$$\min \quad (D^{UB} + 1)R + \sum_{p \in \mathcal{P}} D_p.$$

The coefficient $D^{UB} + 1$ makes the range more important than all possible absolute deviations combined. Therefore the solver first minimizes the difference between the most and least innings played. Among schedules with that range, it minimizes total distance from the equal share.

5.2 Stage 2: positional consistency

After Stage 1, the best range and total deviation found are fixed as constraints. Stage 2 cannot worsen them. It minimizes

$$\begin{aligned} 1000 \sum_{i,p,s} q_{ips}^{(1)} + 200 \sum_p e_p + 25 \sum_{i,p,s} q_{ips}^{(2)} \\ + 5 \sum_p u_p + \sum_{i,p,s} a_{ips}. \end{aligned}$$

The weights express managerial preferences. A one-inning stint receives the largest penalty. Using more than two distinct positions receives an additional large penalty. Exact two-inning stints receive a smaller penalty, which means three- or four-inning assignments are preferred when all else is equal. The last two terms reduce the total number of positions and the number of times position stints begin. Consequently, long stable assignments tend to win remaining ties.

These are soft costs, not absolute rules. This distinction is important. With specialists and strict gender requirements, eliminating every short stint can be impossible without worsening playing-time fairness. The model returns the best compromise it can find while never violating a hard rule.

6 Solution Algorithm

The model is implemented in Python with the OR-Tools CP-SAT solver. CP-SAT is intended for optimization models made from integer and Boolean variables [1]. It combines constraint-programming ideas with Boolean satisfiability methods. At a high level, it repeatedly propagates constraints to rule out impossible assignments, branches on undecided variables, learns from contradictions, and uses objective bounds to discard branches that cannot improve the current solution. This is much more efficient than explicitly listing every possible seven-inning schedule.

The application follows the procedure below.

1. Read the roster, gender, availability, and preferences. Remove unavailable players and reject blank or duplicate names.
2. Determine K , $\mathcal{S}_K$, g_K , and every eligibility set E_p . Run simple infeasibility checks before building the model.
3. Create assignment variables and all hard constraints. Add the auxiliary fairness and consistency variables.
4. Solve the Stage 1 fairness objective. With the current three-second overall limit, this phase receives up to about 0.75 seconds.
5. Fix the range and total deviation found in Stage 1. Pass its assignment as a solution hint, which gives Stage 2 a useful starting point.
6. Use the remaining time to minimize the consistency penalty. The solver uses up to eight worker threads and a fixed random seed.
7. If the status is *OPTIMAL* or *FEASIBLE*, translate the binary variables into a table. Otherwise, show an infeasibility or time-limit message.

The time limit is a practical design choice. Assignment and scheduling problems are combinatorial: adding players, innings, and eligible positions can increase the search space very quickly. A game manager values a high quality answer now more than a proof of optimality several minutes later. Therefore a result marked *FEASIBLE* is acceptable; it satisfies every hard rule but the solver has not proved that no better objective value exists. The website reports both solver status and elapsed time.

7 Implementation, Output, and Limitations

The website loads default names, genders, and preferences from `roster_positions.csv`. All players initially appear as available. Users tap names to change availability and can edit preferences through individual player cards. Because Streamlit keeps widget state separately for each browser session, several low-volume users can run independent schedules without a database.

The main output is a matrix with seven inning rows and the ten standard position columns. Inactive positions contain a dash. A final *Out* column lists the available players sitting in that inning, separated by commas. A second table reports total innings and positions by player, and the complete matrix can be downloaded as a CSV file. Automated tests cover eight-, nine-, and ten-player rules; the RF-to-RC exception; hard preferences; gender placement; playing-time counts; position continuity; CSV defaults; and the Streamlit interface.

The current model balances playing time only within one game. It does not remember who sat more often in earlier games, and it assumes availability is constant for all seven innings. Preferences are also unranked: every selected position is equally acceptable. A future version could store season totals, allow late arrival or early departure, distinguish “preferred” from “allowed,” or let a manager adjust objective weights. These additions would require more input data and possibly persistent storage, but the same binary assignment framework could support them.

8 Conclusion

The softball scheduling problem is a useful example of optimization because it combines strict rules with human-centered preferences. Binary variables represent who plays each position, linear constraints guarantee legal lineups, and a two-stage objective gives clear priority to fairness before positional

stability. CP-SAT can search the resulting combinatorial space quickly enough for last-minute roster changes. The deployed website turns the mathematical model into a practical tool: a manager changes availability, clicks one button, and receives a legal, fair, and reasonably consistent seven-inning schedule.

A Deployment to Streamlit Community Cloud

The service was deployed to *Streamlit Community Cloud*. Streamlit was chosen because it hosts small Python data applications without requiring a separately managed web server, container cluster, or database. Community Cloud is available as a free hosting service and connects directly to GitHub repositories [2].

The deployment used the following steps:

1. The local Python project was placed under Git version control. The root contains `app.py`, the optimizer package, `roster_positions.csv`, `requirements.txt`, and the `.streamlit/config.toml` theme configuration.
2. A GitHub repository was created at <https://github.com/davidluizrusso/softball-fielding>. The repository was made public so Streamlit could read it and the final app could be used by team members without requiring a login.
3. Python dependencies were declared in `requirements.txt`: Streamlit for the interface, pandas for tabular data, and OR-Tools for CP-SAT. Community Cloud uses this file to recreate the Python environment [3].
4. In the Community Cloud dashboard, *Create app* was selected. The source was the `main` branch and the entry point was `app.py`. The equivalent GitHub file URL is <https://github.com/davidluizrusso/softball-fielding/blob/main/app.py>. Streamlit then installed the dependencies and launched the application [4].
5. The app's sharing setting was made public. Streamlit assigned a `streamlit.app` URL that can be opened in Chrome or Safari on desktop and mobile devices without an account.

No secrets or database credentials are required. Roster changes made in the page stay in that visitor's session, while the checked-in CSV provides the default roster for a new session. Code updates follow a simple workflow: modify and test locally, commit the changes, and run `git push`. Community Cloud watches the GitHub repository and redeploys pushed changes automatically [5]. This approach meets the project's goals of low cost, low administration, and a short path from local code to a shareable website.

References

- [1] Google for Developers. *CP-SAT Solver: OR-Tools*. Accessed July 26, 2026.
- [2] Streamlit. *Streamlit Community Cloud*. Accessed July 26, 2026.
- [3] Streamlit. *App Dependencies for Your Community Cloud App*. Accessed July 26, 2026.
- [4] Streamlit. *Deploy Your App on Community Cloud*. Accessed July 26, 2026.
- [5] Streamlit. *Manage Your App*. Accessed July 26, 2026.